Welcome to the Python Course!

This document compiles the advanced topics learned during the second week. It serves as a comprehensive study guide, detailing key concepts, syntax, and basic practical examples. All explanations and code blocks are derived directly from the weekly log repository.

Table of Contents / Topics Covered:

1. Functions

On Day 7, we introduced functions in Python. We covered function definitions, function calls, parameters, arguments, return statements, and walked through several illustrative examples.

1. What is a Function?

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

■ 2. Function Definition & Function Call

Function Definition

To define a function, use the `def` keyword followed by the function name and parentheses `()`. Any input parameters or arguments should be placed within these parentheses.

```
def greet(text_message):
    print(text_message)
```

Function Call

To execute the function, call it by its name followed by parentheses containing the required arguments.

```
greet(54)
```

---

3. Parameters vs. Arguments

- **Parameters**: The variables listed inside the parentheses in the function definition. They act as placeholders for the data the function needs.
- **Arguments**: The actual values passed to the function when it is called.

---

4. Return Statements

The `return` statement is used to exit a function and pass a value back to the caller.

```python
def add(a, b):
    s = a + b
    return s

sum1 = add(1, 2)
sum2 = add(sum1, 3)
print(sum2)  # Output: 6
```

5. Practical Examples Covered

1■■ Count Positive Numbers in a List

A function that counts and returns the number of positive elements in a list.

```python
def countPos(user_list):
    pos = 0
    for i in user_list:
        if i > 0:
            pos += 1
    return pos

l1 = [0, -1, 1, 2, 3]
l2 = [0, -1, -2, 3, 5]

pos_l1 = countPos(l1)
pos_l2 = countPos(l2)

if pos_l1 == pos_l2:
    print("Same Pos Nums")
else:
    print("No")
```

2■■ Check Even or Odd (with User Input)

A function that checks if a number is even or odd, takes input from the user, and prints the result.

```python
def even_odd(val):
    if val % 2 == 0:
        return f"Number {val} is even"
    else:
        return f"Number {val} is odd"

val1 = int(input("Enter the number: "))
result = even_odd(val1)
print(result)
```

2. Scope & Function Annotations

On Day 8, we covered variable scopes (local vs. global), function annotations (type hints), and walked through two basic practical examples: Palindrome Checking and Factorial Computation.

1. Local vs. Global Variables

In Python, the scope of a variable refers to the region of the program where a variable is recognized and can be accessed.

Local Variables

Variables defined inside a function are local to that function. They are created when the function starts executing and are destroyed when the function returns. They cannot be accessed from outside the function.

```
def my_function():
    x = 10  # Local variable
    print("Inside function:", x)

my_function()
# print(x)  # This will raise a NameError because x is local to my_function
```

Global Variables

Variables defined outside of any function are global variables. They can be accessed from anywhere within the program, including inside functions (for reading).

```
y = 20  # Global variable

def print_y():
    print("Inside function:", y)  # Accessing global variable

print_y()
print("Outside function:", y)
```

Modifying Global Variables (`global` Keyword)

To modify a global variable inside a function, you must declare it using the **`global`** keyword.

```
counter = 0  # Global variable

def increment():
    global counter  # Declare intention to modify the global variable
    counter += 1

increment()
print(counter)  # Output: 1
```

■ 2. Function Annotations

Function annotations are optional metadata about the types used by user-defined functions. They act as **type hints** to specify what type of arguments a function expects and what type it returns. Python ignores them at runtime, but they are highly useful for documentation and static analysis tools.

■ Syntax:

- **Parameter Annotations**: `parameter: type`
- **Return Annotation**: `-> type`

```
# Annotation indicates name is a string, age is an integer, and the function returns a string
def greet(name: str, age: int) -> str:
    return f"Hello {name}, you are {age}."
```

3. Basic Examples Covered

1■■ Palindrome Checking

A basic example demonstrating string slicing to check if a word is a palindrome, using function annotations.

```
# Function with type annotations
def palindromeChecker(word: str) -> bool:
    # 'word' is a local variable/parameter
    is_palindrome = (word == word[::-1])  # 'is_palindrome' is a local variable
    return is_palindrome

# 'userInput' is a global variable
userInput = input("Enter a word: ")
if palindromeChecker(userInput):
    print("Yes it&#x27;s a palindrome")
else:
    print("No it&#x27;s not a palindrome")
```

2■■ Compute Factorial

A function to compute the factorial of a number using a loop, demonstrating local variable accumulator and annotations.

```python
# Function with type annotations.
# Note: In our script, we return a string message if input is invalid,
# so the return type can be either int or str.
def computeFactorial(n: int) -> int | str:
    # 'n' is a local parameter
    if n < 0:
        return "Please enter a value greater than or equal to 0"

    fact = 1  # 'fact' is a local variable
    for i in range(1, n + 1):  # 'i' is a local loop variable
        fact *= i

    return fact

# 'n' is a global variable in the main block
n = int(input("Enter a number: "))
print(f"Factorial of {n} is: {computeFactorial(n)}")
```

3. Nested Functions & ATM Simulation

On Day 9, we covered **nested functions** (also known as **inner functions**), their scope, and implemented an ATM simulation program to demonstrate how multiple nested functions can be structured inside a single parent function.

1. What is a Nested Function?

A **nested function** is a function defined inside another function. In Python, functions are first-class citizens, meaning they can be defined anywhere, passed as arguments, and returned from other functions.

Syntax

```
def outer_function():
    # Outer function scope
    def inner_function():
        # Inner function scope
        pass
    inner_function()  # Call the inner function inside the outer function
```

Key Characteristics:

1. **Encapsulation & Information Hiding**: The inner function is not accessible from the global scope. It is hidden and only exists within the scope of the outer function.
2. **Access to Outer Scope**: Inner functions can read variables defined in the outer function.

---

2. ATM Simulation Example

To demonstrate nested functions, we implemented an ATM simulation program. In this example, the main function `atm()` contains three nested functions:

- `validateUser`: Validates the PIN entered by the user.
- `showBalance`: Displays the current balance.
- `withdraw`: Validates the withdrawal amount and prints the transaction status.

Python Code (day_09.py)

```python
def atm():
  balance = 1000

  def validateUser(pin : int) -> bool:
    """Check whether pin is correct."""
    correct_pin = 2005

    if pin == correct_pin:
      return True
    else:
      return False

  def showBalance(balance : int) -> None:
    """Display current balance of the user."""
    print(f"Current Balance: {balance}")

  def withdraw(amount : int, balance : int) -> None:
    """Perform the withdraw operation"""
    if amount < 0:
      print("Amount cannot be negative.")
      return

    if amount > balance:
      print("Not enough funds.")
      return

    print("Please collect your cash.")
    balance -= amount
    showBalance(balance)

  pin = int(input("Enter your PIN: "))
  status = validateUser(pin)
  if status is False:
    print("PIN incorrect. Please try again.")
    return

  print("Authentication completed.")
  showBalance(balance)

  amount = int(input("Please enter amount greater than 0: "))
  withdraw(amount, balance)

atm()
```

Scope & Design Considerations:

- **Encapsulation**: The helper functions `validateUser`, `showBalance`, and `withdraw` are defined inside `atm()`, preventing them from polluting the global namespace.
- **Parameters**: `balance` is passed directly to `showBalance` and `withdraw` as an argument. Since integers are immutable in Python, assigning a new value to `balance` inside `withdraw` (via `balance -= amount`) only

changes it inside the local scope of `withdraw`. It does not modify the `balance` variable in the outer `atm()` function. If we needed to modify the outer scope variable directly without passing parameters, we could use the `nonlocal` keyword.

4. Recursion & Factorial

On Day 10, we covered **recursion**, including the concepts of a base case, recursive case, execution flow, the call stack, and analyzed a simple recursive **factorial** function.

1. What is Recursion?

Recursion is a programming method where a function calls itself, either directly or indirectly, to solve a problem by dividing it into smaller subproblems of the same type.

A proper recursive function contains two essential components:

- **Base Case**: The termination condition under which the function stops calling itself and starts returning values. This prevents infinite recursion and stack overflow errors (`RecursionError` in Python).
- **Recursive Case**: The code block where the function makes a recursive call to itself with a modified, usually smaller or simpler input, progressing toward the base case.

---

2. The Recursive Stack & Flow

When a recursive function is called, Python allocates a new frame on the **call stack** to manage its execution context (local variables, arguments, and return address).

1. **Winding (Pushing)**: The function continuously calls itself, pushing a new frame onto the stack for each call, until the base case is met.
2. **Unwinding (Popping)**: Upon hitting the base case, the function starts returning values. The stack frames are popped one by one, resolving the pending computations in reverse order until the original function call completes.

Recursive Stack Flow for `factorial(5)`:

```
factorial(5)  -> waits for factorial(4)
  factorial(4)  -> waits for factorial(3)
    factorial(3)  -> waits for factorial(2)
      factorial(2)  -> waits for factorial(1)
        factorial(1) -> returns 1 (Base Case reached)
      factorial(2)  -> returns 2 * 1 = 2
    factorial(3)  -> returns 3 * 2 = 6
  factorial(4)  -> returns 4 * 6 = 24
factorial(5)  -> returns 5 * 24 = 120
```

---

3. Simple Factorial Recursive Example

A classic example of recursion is computing the factorial of a number n (written as $n!$).

- **Mathematical Definition**: $n! = n \times (n-1)!$ for $n > 1$, and $1! = 1$ (the base case).

Python Code (day_10.py)

```python
# iteration
# n = 5
# pdt = 1
# for i in range(1,n+1):
#   pdt *= i
# print(pdt)

# recursion
def factorial(n):
  # base case - ends the recursion
  if n == 1:
    return 1
  # recursive case
  return n * factorial(n-1)

print(factorial(5))
```

Practice Questions

Part A: Functions

Question 1: Greeting Function

Write a function **greet(name)** that takes a person's name as input and prints a greeting message.

```
Example
Input:
Vijay

Output:
Hello, Vijay! Welcome!
```

Question 2: Sum of Two Numbers

Write a function **add(a, b)** that takes two integers as arguments and returns their sum.

```
Example
Input:
12 8

Output:
20
```

Question 3: Maximum of Three Numbers

Write a function **maximum(a, b, c)** that takes three numbers as input and returns the largest among them.

```
Example 1
Input:
10 25 18

Output:
25

Example 2
Input:
50 12 30

Output:
50
```

Question 4: Square of Two Numbers

Write a function **square(a, b)** that returns the squares of both numbers.

```
Example
Input:
4 7

Output:
16 49
```

Bonus: Write another function that calculates the sum of these two squares.

Question 5: Count the Vowels in a String

Write a function **count_vowels(text)** that returns the total number of vowels (a, e, i, o, u) present in the given string.

```
Example
Input:
Artificial Intelligence

Output:
10
```

Part B: Recursion

Question 6: Print Numbers from n to 0

Write a recursive function **countdown(n)** that prints numbers from n down to 0.

```
Example
Input:
5

Output:
5
4
3
2
1
0
```

Question 7: Sum of First n Natural Numbers

Write a recursive function **sum_n(n)** that returns the sum of the first n natural numbers.

```
Example
Input:
5

Output:
15
```

Explanation: 5 + 4 + 3 + 2 + 1 = 15

Bonus Challenge

Question 8: Recursive Greeting

Write a recursive function that prints "Hello" exactly n times.

```
Example
Input:
3

Output:
Hello
Hello
Hello
```

Simulation Question

Problem Statement

Design a Python program to simulate a simple Restaurant Ordering System using functions and nested functions.

Write a function named **restaurant()** that acts as the main function. Inside **restaurant()**, define the following nested functions:

- **show_menu()** – Displays the available food items along with their prices.
- **take_order()** – Accepts the customer's food choice.
- **calculate_bill()** – Calculates and displays the total bill based on the selected items.

Menu

Item	Price (Rs.)
Burger	100
Pizza	200
Juice	50

Requirements

- Display the menu to the customer.
- Allow the customer to select one or more food items.
- Assume that the quantity of every selected item is always 1.
- Calculate the total bill amount.
- Display the final bill showing: Item name, Price of each item, and Total bill amount.

Sample Run

```
Input

Welcome to ABC Restaurant

Menu
1. Burger - Rs. 100
2. Pizza - Rs. 200
3. Juice - Rs. 50

How many items would you like to order? 2

Enter item number: 1

Enter item number: 3

Output

--------- BILL ---------
Burger - Rs. 100
Juice - Rs. 50
------------------------
Total Bill = Rs. 150

Thank you for visiting ABC Restaurant!
```